RETAIL BILLING WITH MULTIPLE DISCOUNT RULES

Python Activity-Based Learning (ABL) Assignment — Sample Implementation

1. Problem Statement

Develop a Python program that processes multiple products and quantities, applies category-specific taxes, calculates threshold-based discounts, enforces a maximum concession limit, and generates an itemized invoice.

2. Objectives

• Accept and process multiple product records.

• Calculate each line amount from unit price × quantity.

• Apply a threshold discount to the pre-tax subtotal.

• Cap the discount at the maximum concession of Rs. 2,000.

• Apply different tax rates by product category and print an itemized invoice.

3. Assumptions and Business Rules

Rule	Value used in this sample
Grocery tax	5%
Clothing tax	12%
Electronics tax	18%
Subtotal ≥ Rs. 1,000	5% discount
Subtotal ≥ Rs. 5,000	10% discount
Subtotal ≥ Rs. 10,000	15% discount
Maximum concession	Rs. 2,000

4. Algorithm

1. Start the program.

2. Store the product details: name, category, quantity, and unit price.

3. Calculate the pre-tax subtotal from all line amounts.

4. Select the discount rate using the subtotal thresholds.

5. Calculate discount = minimum(subtotal × rate, maximum concession).

6. Allocate the discount proportionally to each product line.

7. Calculate category tax on each discounted line amount.

8. Add discounted amounts and taxes to obtain the grand total.

9. Print the itemized invoice and stop.

5. Flowchart

```
START
  |
Read / define product records
  |
Calculate subtotal
```

```
  |
Choose threshold discount rate
  |
Discount = min(subtotal × rate, Rs. 2,000)
  |
For each item: allocate discount → apply category tax
  |
Calculate grand total
  |
Print itemized invoice
  |
 STOP
```

6. Pseudocode

```
subtotal = sum(price * quantity for every product)
rate = 15% if subtotal >= 10000
       else 10% if subtotal >= 5000
       else 5% if subtotal >= 1000
       else 0%
discount = min(subtotal * rate, 2000)
for each product:
    line_discount = (line_amount / subtotal) * discount
    taxable_amount = line_amount - line_discount
    tax = taxable_amount * category_tax_rate
    line_total = taxable_amount + tax
grand_total = subtotal - discount + sum(all taxes)
print itemized invoice
```

7. Python Source Code

The complete runnable source file is also provided separately as **retail_billing.py**. The code below contains the implementation used to generate the sample output.

```
"""
Retail Billing with Multiple Discount Rules
Python Activity-Based Learning (ABL) Assignment

Rules:
- Category-specific tax rates: Grocery 5%, Clothing 12%, Electronics 18%
- Threshold discount on subtotal: 5% for >= Rs. 1,000;
  10% for >= Rs. 5,000; 15% for >= Rs. 10,000
- Maximum concession (discount) is capped at Rs. 2,000.
- Discount is allocated proportionally to items before calculating tax.
"""

TAX_RATES = {
    "grocery": 0.05,
    "clothing": 0.12,
    "electronics": 0.18,
}

MAX_CONCESSION = 2000.00


def get_discount_rate(subtotal):
    """Return the discount rate based on the pre-tax subtotal."""
    if subtotal >= 10000:
        return 0.15
    elif subtotal >= 5000:
        return 0.10
    elif subtotal >= 1000:
        return 0.05
    return 0.0


def calculate_bill(items):
    """Calculate item totals, proportionally allocated discount, category tax, and grand total."""
    subtotal = sum(item["price"] * item["quantity"] for item in items)
    discount_rate = get_discount_rate(subtotal)
    discount = min(subtotal * discount_rate, MAX_CONCESSION)

    invoice_items = []
    for item in items:
        gross = item["price"] * item["quantity"]
        item_discount = (gross / subtotal * discount) if subtotal else 0.0
        taxable_amount = gross - item_discount
        tax_rate = TAX_RATES[item["category"].lower()]
        tax = taxable_amount * tax_rate
        invoice_items.append({
            "name": item["name"],
            "category": item["category"].title(),
            "quantity": item["quantity"],
            "unit_price": item["price"],
            "gross": gross,
            "discount": item_discount,
            "tax_rate": tax_rate,
            "tax": tax,
            "total": taxable_amount + tax,
        })

    total_tax = sum(row["tax"] for row in invoice_items)
    grand_total = subtotal - discount + total_tax
    return {
        "items": invoice_items,
        "subtotal": subtotal,
        "discount_rate": discount_rate,
        "discount": discount,
        "total_tax": total_tax,
        "grand_total": grand_total,
    }

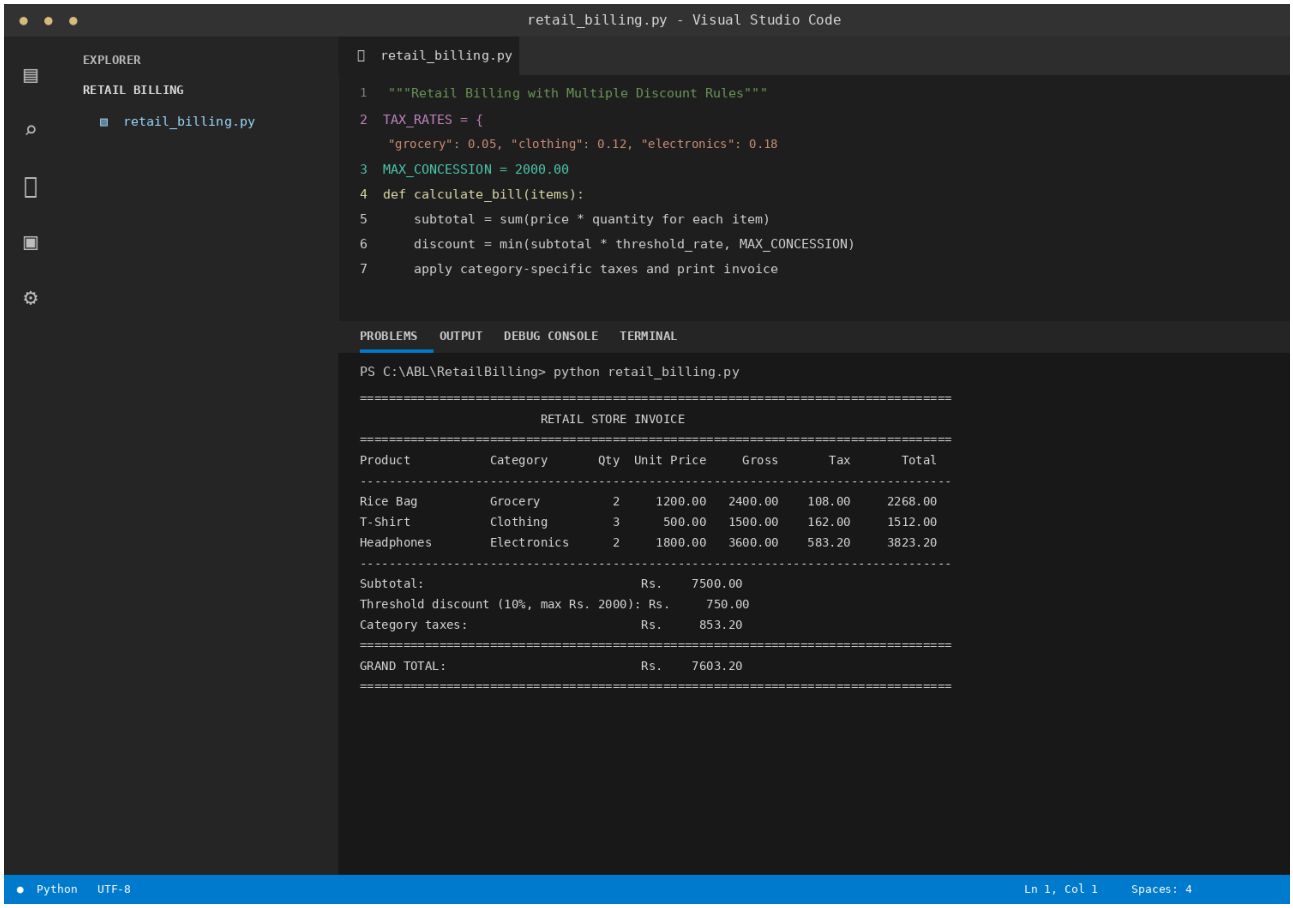

def print_invoice(bill):
    print("=" * 82)
    print("                         RETAIL STORE INVOICE")
    print("=" * 82)
    print(f"{'Product':<18}{'Category':<14}{'Qty':>4}{'Unit Price':>12}{'Gross':>10}{'Tax':>10}{'Total':>12}")
    print("-" * 82)
    for row in bill["items"]:
        print(
            f"{row['name']:<18}{row['category']:<14}{row['quantity']:>4}"
            f"{row['unit_price']:>12.2f}{row['gross']:>10.2f}"
            f"{row['tax']:>10.2f}{row['total']:>12.2f}"
        )
    print("-" * 82)
    print(f"Subtotal:                              Rs. {bill['subtotal']:>10.2f}")
    print(f"Threshold discount ({bill['discount_rate'] * 100:.0f}%, max Rs. {MAX_CONCESSION:.0f}): Rs. {bill['discount']:>10.2f}")
```

```
    print(f"Category taxes:                        Rs. {bill['total_tax']:>10.2f}")
    print("=" * 82)
    print(f"GRAND TOTAL:                           Rs. {bill['grand_total']:>10.2f}")
    print("=" * 82)


if __name__ == "__main__":
    # Sample test data for a reproducible demonstration.
    products = [
        {"name": "Rice Bag", "category": "grocery", "quantity": 2, "price": 1200.00},
        {"name": "T-Shirt", "category": "clothing", "quantity": 3, "price": 500.00},
        {"name": "Headphones", "category": "electronics", "quantity": 2, "price": 1800.00},
    ]
    result = calculate_bill(products)
    print_invoice(result)
```

8. Test Data and Executed Output

Sample products: Rice Bag (Grocery, quantity 2, Rs. 1,200 each), T-Shirt (Clothing, quantity 3, Rs. 500 each), Headphones (Electronics, quantity 2, Rs. 1,800 each). The screenshot below is a VS Code-style presentation of the output produced by executing the Python file in this environment.



Terminal output (text copy)

```
================================================================================
                        RETAIL STORE INVOICE
================================================================================
Product           Category       Qty  Unit Price     Gross       Tax       Total
--------------------------------------------------------------------------------
Rice Bag          Grocery          2     1200.00   2400.00    108.00     2268.00
T-Shirt           Clothing         3      500.00   1500.00    162.00     1512.00
Headphones        Electronics      2     1800.00   3600.00    583.20     3823.20
--------------------------------------------------------------------------------
Subtotal:                                 Rs.    7500.00
Threshold discount (10%, max Rs. 2000): Rs.     750.00
Category taxes:                           Rs.     853.20
================================================================================
GRAND TOTAL:                              Rs.    7603.20
================================================================================
```

9. Test Cases

Test	Subtotal	Expected discount rule	Purpose
1	Rs. 800	0%	Below first threshold
2	Rs. 1,000	5%	First threshold boundary
3	Rs. 5,000	10%	Second threshold boundary

Test	Subtotal	Expected discount rule	Purpose
4	Rs. 10,000	15%	Third threshold boundary
5	Rs. 20,000	Rs. 2,000 cap	Maximum concession limit

10. Conclusion

The program demonstrates multi-product billing, quantity-based line totals, category-specific taxes, subtotal threshold discounts, a maximum concession cap, and itemized invoice generation. Tax percentages and discount policies are sample assumptions; replace them with the rules specified by your faculty or retailer if they differ.